SISTEMA DE VENDAS · VENDAS-WEB

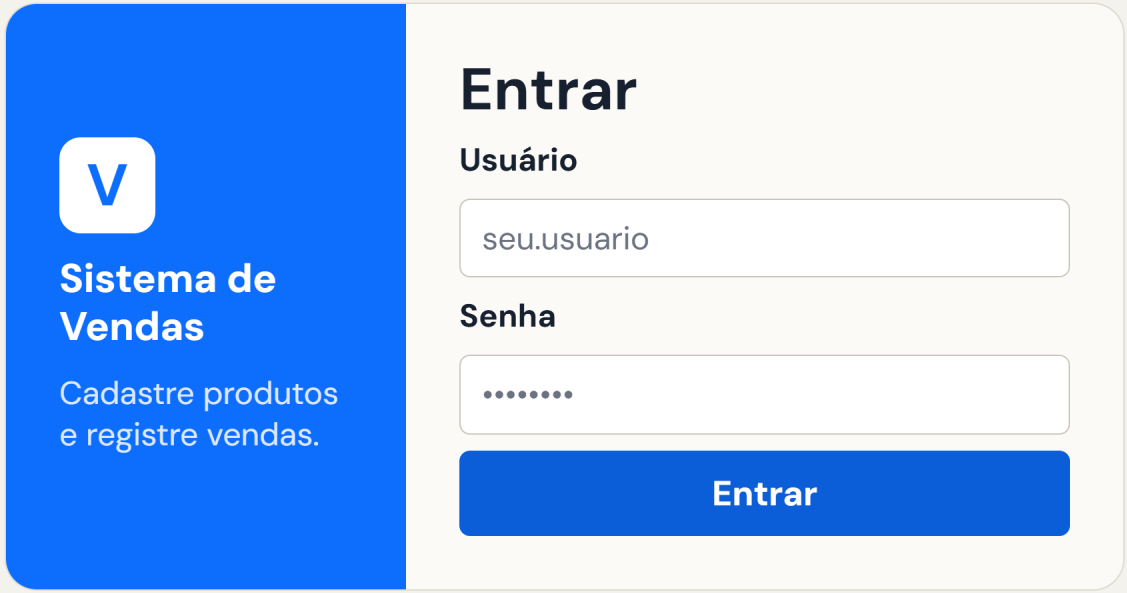
Tela de Login com Vue.js

Construindo passo a passo: implementar, testar e entender.



VISÃO GERAL

O que vamos construir



Ao final você terá

- Uma tela de login completa em Vue 3
- Validação e mensagens de erro na tela
- Sessão guardada no navegador
- Rotas protegidas por um guard
- Login real conversando com a API Spring

Método: implementar, testar, entender

ROTEIRO

Sete etapas, cada uma com teste

Etapa	O que construímos	Arquivos	Como testamos
1	Tela, estilo, rota e RouterView	LoginView.vue, App.vue	Abrir /login no navegador
2	Estado com ref e v-model	LoginView.vue	Digitar e ver o console
3	Mensagem de erro na tela	LoginView.vue	Enviar com campos vazios
4	Função login() e botão	services/auth.js	admin/admin123 certo e errado
5	Guardar usuário, Home e Sair	stores/session.js	Entrar, F5 e Session Storage
6	Guard que bloqueia sem login	router/index.js	Abrir / sem estar logado
7	fetch para a API Spring	services/auth.js	Entrar com o usuário do banco

Em cada etapa: escrevemos pouco código, testamos e só então explicamos o que aconteceu.

ETAPA 1 DE 7

Tela visível

Primeiro a tela aparece no navegador, ainda sem nenhuma lógica.

Pronto quando: /login mostra a tela em duas colunas

CONCEITOS DA ETAPA 1

Uma tela inteira em um arquivo

Componente .vue

Um arquivo reúne o template (HTML), o script (JavaScript) e o style (CSS) de uma mesma tela.

style scoped

O CSS vale só para este componente. Os estilos da tela de login não vazam para as outras telas.

Bootstrap

Biblioteca de classes CSS prontas, como form-control e btn. Já está carregada por CDN no index.html.

CSS Grid

Divide a tela em colunas. O valor 1fr 1fr cria duas colunas de tamanhos iguais.

ETAPA 1 · LOGINVIEW.VUE (1 DE 2)

A estrutura da tela

NOVO ARQUIVO · src/views/LoginView.vue

```
<template>
  <div class="login">
    <section class="login__brand">
      <div class="login__logo">V</div>
      <h1>Sistema de Vendas</h1>
      <p>Cadastre produtos e registre vendas.</p>
    </section>
    <section class="login__form">
      <!— o formulário entra aqui (próximo slide) —>
    </section>
  </div>
</template>
```

login é a grade de duas colunas. brand é o lado azul. form é o lado do formulário.

ETAPA 1 · LOGINVIEW.VUE (2 DE 2)

O formulário

DENTRO DE <section class="login__form">

```
<form class="login__box" novalidate>
  <h2>Entrar</h2>
  <label for="usuario" class="form-label">Usuário</label>
  <input id="usuario" type="text" class="form-control mb-3" />
  <label for="senha" class="form-label">Senha</label>
  <input id="senha" type="password" class="form-control mb-4" />
  <button type="submit" class="btn btn-primary w-100">Entrar</button>
</form>
```

label for=id liga o texto ao campo: clicar no texto foca o input.

type="password" esconde o que é digitado.

form-control, btn são classes do Bootstrap.

ETAPA 1 · ESTILO (1 DE 2)

Duas colunas com CSS Grid

LOGINVIEW.VUE · DEPOIS DO </template>

```
<style scoped>
.login {
  min-height: 100vh;
  display: grid;
  grid-template-columns: 1fr 1fr;
}
.login__form {
  display: flex; align-items: center; justify-content: center;
}
.login__box { width: 100%; max-width: 480px; }
```

100vh ocupa a altura da janela.

1fr 1fr cria duas colunas iguais.

flex + center centraliza o formulário.

ETAPA 1 · ESTILO (2 DE 2)

O lado azul e o logo

CONTINUA NO BLOCO STYLE

```
.login__brand {
  background: #0d6efd; color: #fff; padding: 4rem;
  display: flex; flex-direction: column; justify-content: center;
}
.login__logo {
  width: 96px; height: 96px; border-radius: 20px;
  background: #fff; color: #0d6efd; font-size: 3rem; font-weight: 700;
  display: grid; place-items: center; margin-bottom: 2rem;
}
</style>
```

rem é relativo à fonte: 4rem equivale a 64px.

flex column empilha logo, título e texto.

place-items centraliza o V no quadrado.

ETAPA 1 · ROUTER/INDEX.JS

Registrando a rota /login

ADICIONE AO LADO DAS ROTAS QUE JÁ EXISTEM

```
// dentro de routes: [ ... ]
{
  path: '/login',
  name: 'login',
  component: () ⇒ import('@/views/LoginView.vue'),
},
```

path é o endereço que aparece na barra do navegador.

name é um apelido, usado nos redirecionamentos.

import() carrega a tela só quando a rota é visitada.

ETAPA 1 · APP.VUE

Onde a rota aparece: RouterView

1 · A URL
O navegador abre /login.

2 · O router
Escolhe o LoginView.vue.

3 · O RouterView
Desenha a tela escolhida.

SUBSTITUA TODO O CONTEÚDO DE src/App.vue

```
<template>
  <RouterView />
</template>
```

Sem o RouterView a rota existe, mas nada aparece: o App.vue de exemplo só mostra o “You did it!”.

TESTE DA ETAPA 1

Hora de ver no navegador

Faça

1. Rode npm run dev
2. Abra localhost:5173/login
3. Digite algo nos dois campos
4. Clique em Entrar

Esperado

- Duas colunas: lado azul e formulário
- A senha aparece como bolinhas
- Ao clicar em Entrar a página recarrega

A página recarregar é o comportamento padrão de um formulário HTML. É exatamente isso que vamos controlar na etapa 2.

ETAPA 2 DE 7

Estado do formulário

O que o usuário digita passa a existir como dado dentro do Vue.

Pronto quando: o console mostra o que foi digitado

CONCEITOS DA ETAPA 2

Reatividade em quatro ideias

ref()

Cria um dado reativo. Quando o valor muda, tudo na tela que o usa é atualizado sozinho.

.value

No script lemos e gravamos com .value (usuario.value). No template usamos só o nome (usuario).

v-model

Ligação de duas vias: o que se digita atualiza a variável, e mudar a variável atualiza o campo.

@submit.prevent

Escuta o envio do formulário e cancela o recarregamento padrão da página.

ETAPA 2 · LOGINVIEW.VUE (1 DE 2)

O script: estado e função

NO TOPO DO ARQUIVO, ANTES DO <template>

```
<script setup>
import { ref } from 'vue'

const usuario = ref('')
const senha = ref('')

function entrar() {
  console.log('usuario:', usuario.value, '| senha:', senha.value)
}
</script>
```

ref('') começa com texto vazio.

entrar() por enquanto só escreve no console.

Atenção: senha no console é só para a aula.

ETAPA 2 · LOGINVIEW.VUE (2 DE 2)

O template: ligando tudo

ALTERE O FORM E OS DOIS INPUTS

```
<form class="login__box" novalidate @submit.prevent="entrar">
  ...
  <input id="usuario" v-model="usuario" type="text" ... />
  ...
  <input id="senha" v-model="senha" type="password" ... />
  ...
</form>
```

@submit.prevent="entrar" chama entrar() e cancela o recarregamento.

v-model="usuario" liga o campo à variável usuario.

As reticências marcam o que já estava escrito.

TESTE DA ETAPA 2

Ver os dados no console

Faça

1. Abra o DevTools (F12) na aba Console
2. Digite um usuário e uma senha
3. Clique em Entrar

Esperado

- A página não recarrega mais
- O console mostra usuario e senha digitados
- No Vue DevTools os refs mudam enquanto você digita

Já temos dados vivos dentro do componente. Na etapa 3 a tela vai reagir a eles.

ETAPA 3 DE 7

Validação

Antes de falar com qualquer servidor, a tela confere se os campos foram preenchidos.

Pronto quando: campos vazios mostram uma mensagem de erro

CONCEITOS DA ETAPA 3

Decidir o que mostrar

Guard clause

Um if que encerra a função cedo com return. Se faltou dado, nem tentamos continuar.

trim()

Remove espaços das pontas do texto. Sem ele, um campo só com espaços passaria como preenchido.

v-if

Mostra o elemento só quando a condição é verdadeira. Com o erro vazio, o alerta nem existe na tela.

Interpolação {{ }}

Escreve o valor de uma variável dentro do HTML do template.

ETAPA 3 · LOGINVIEW.VUE

Validar e mostrar o erro

SCRIPT · NOVA VARIÁVEL E NOVA VERSÃO DE entrar()

```
const erro = ref('')
function entrar() {
  erro.value = ''
  if (!usuario.value.trim() || !senha.value) {
    erro.value = 'Informe usuário e senha.'
    return
  }
  console.log('Campos preenchidos, pode enviar!')
}
```

TEMPLATE · DEPOIS DO BOTÃO, DENTRO DO FORM

```
<div v-if="erro" class="alert alert-danger mt-3">{{ erro }}</div>
```

TESTE DA ETAPA 3

Campos vazios e preenchidos

Faça

1. Clique em Entrar sem digitar nada
2. Digite só o usuário e clique de novo
3. Preencha os dois campos e clique

Esperado

- Nos dois primeiros casos aparece o alerta vermelho
- Com tudo preenchido o alerta some
- O console mostra a mensagem de campos preenchidos

Nenhuma linha mexeu no HTML. Mudamos a variável erro e a tela se ajustou sozinha.

ETAPA 4 DE 7

Login simulado

Criamos uma função login() de mentira, que finge consultar a API.
Assim testamos o fluxo inteiro antes de ter servidor.

Pronto quando: admin e admin123 entram, o resto mostra erro

CONCEITOS DA ETAPA 4

Código que espera uma resposta

Promise

A promessa de um resultado futuro. Tarefas demoradas, como falar com um servidor, devolvem uma Promise.

async e await

await pausa só esta função até a Promise terminar. O resto da tela continua respondendo.

try, catch, finally

try tenta, catch trata o erro, finally roda sempre, tenha dado certo ou errado.

Camada de serviço

auth.js cuida do login. A tela só chama login() e não precisa saber como ele funciona por dentro.

ETAPA 4 · SERVICES/AUTH.JS

A função login() de teste

NOVO ARQUIVO · src/services/auth.js

```
// Versão de teste: finge consultar a API (1 segundo)
export async function login(usuario, senha) {
  await new Promise((resolve) ⇒ setTimeout(resolve, 1000))

  if (usuario ═══ 'admin' && senha ═══ 'admin123') {
    return { token: 'token-de-teste', usuario: { nome: 'Administrador' } }
  }
  throw new Error('Usuário ou senha inválidos.')
}
```

setTimeout simula a demora da rede.

O retorno já tem o formato da API real.

throw cria um erro que o catch vai capturar.

ETAPA 4 · LOGINVIEW.VUE (1 DE 2)

Estado de carregamento

SCRIPT · NOVO IMPORT E NOVA VARIÁVEL

```
import { login } from '@/services/auth'
const carregando = ref(false)
```

TEMPLATE · SUBSTITUA O BOTÃO

```
<button type="submit" class="btn btn-primary w-100" :disabled="carregando">
  <span v-if="carregando" class="spinner-border spinner-border-sm me-2"></span>
  {{ carregando ? 'Entrando...' : 'Entrar' }}
</button>
```

ref(false) é um interruptor: ligado enquanto espera.

:disabled liga o atributo disabled a uma variável.

? : operador ternário: condição ? se sim : se não.

ETAPA 4 · LOGINVIEW.VUE (2 DE 2)

entrar() com try, catch e finally

NOVA VERSÃO DA FUNÇÃO

```
async function entrar() {
  erro.value = ''
  // ...validação da etapa 3...
  carregando.value = true
  try {
    const dados = await login(usuario.value.trim(), senha.value)
    console.log('Login ok:', dados)
  } catch (e) {
    erro.value = e.message
  } finally {
    carregando.value = false
  }
}
```

finally sempre executa: o botão volta ao normal, tenha dado certo ou errado.

TESTE DA ETAPA 4

Senha errada e senha certa

Faça

1. Entre com admin e uma senha errada
2. Entre com admin e admin123
3. Observe o botão durante a espera

Esperado

- Por 1 segundo: spinner e Entrando...
- Senha errada: alerta Usuário ou senha inválidos.
- Senha certa: console com Login ok e o token

O login já funciona de ponta a ponta. Falta guardar quem entrou e levar essa pessoa a outra tela: etapa 5.

ETAPA 5 DE 7

Sessão e Home

Guardamos quem entrou, criamos a tela Home e o botão Sair.

Pronto quando: depois do F5 a Home continua mostrando o seu nome

CONCEITOS DA ETAPA 5

Lembrar quem entrou

Token

Comprovante emitido pelo servidor depois do login. Na API real é um JWT, que prova que a pessoa já se identificou.

Estado compartilhado

Um arquivo JavaScript com reactive é importado por várias telas, que enxergam o mesmo objeto session.

sessionStorage

Guarda texto no navegador. Sobrevive ao F5 e some quando a aba é fechada.

push x replace

push adiciona uma entrada ao histórico. replace troca a atual, então o botão Voltar não retorna ao login.

ETAPA 5 · STORES/SESSION.JS (1 DE 2)

O objeto session

NOVO ARQUIVO · src/stores/session.js

```
import { reactive } from 'vue'

const salvo = JSON.parse(sessionStorage.getItem('sessao')) ?? {}

export const session = reactive({
  token: salvo.token ?? null,
  usuario: salvo.usuario ?? null,
})
```

reactive é como o ref, mas para objetos.

getItem + JSON.parse leem o texto salvo e o viram objeto.

?? usa o valor da direita quando o da esquerda é null.

ETAPA 5 · STORES/SESSION.JS (2 DE 2)

Iniciar e encerrar a sessão

CONTINUA NO MESMO ARQUIVO

```
export function iniciarSessao({ token, usuario }) {
  session.token = token
  session.usuario = usuario
  sessionStorage.setItem('sessao', JSON.stringify({ token, usuario }))
}

export function encerrarSessao() {
  session.token = null
  session.usuario = null
  sessionStorage.removeItem('sessao')
}
```

Mantemos duas cópias iguais: a reativa (memória) e a persistente (sessionStorage).

ETAPA 5 · VIEWS/HOMEVIEW.VUE (1 DE 2)

Home: o script com o Sair

NOVO ARQUIVO · src/views/HomeView.vue

```
<script setup>
import { useRouter } from 'vue-router'
import { session, encerrarSessao } from '@/stores/session'

const router = useRouter()

function sair() {
  encerrarSessao()
  router.replace('/login')
}
</script>
```

useRouter dá acesso ao roteador pelo código.

encerrarSessao() apaga token e usuário.

replace troca a página atual no histórico.

ETAPA 5 · VIEWS/HOMEVIEW.VUE (2 DE 2)

Home: o template

DEPOIS DO </script>

```
<template>
  <main class="container py-4">
    <h1>Vendas Web</h1>
    <p>Olá, {{ session.usuario?.nome }}</p>
    <button class="btn btn-outline-secondary" @click="sair">Sair</button>
  </main>
</template>
```

?. evita erro quando usuario é null.

@click é a forma curta de v-on:click.

container, py-4 são classes do Bootstrap.

ETAPA 5 · ROUTER/INDEX.JS

Registrando a rota da Home

ADICIONE AO LADO DA ROTA /login

```
// dentro de routes: [ ... ]
{
  path: '/',
  name: 'home',
  component: () ⇒ import('@/views/HomeView.vue'),
},
```

Mesmo formato da rota /login, com outro path e outro componente.

Agora o replace('/') do login tem para onde ir.

Sem esta rota o console avisa e a tela fica em branco.

ETAPA 5 · LOGINVIEW.VUE

O login guarda a sessão e segue

SCRIPT · NOVOS IMPORTS E NOVA CONSTANTE

```
import { useRouter } from 'vue-router'
import { iniciarSessao } from '@/stores/session'
const router = useRouter()
```

DENTRO DO try · NO LUGAR DO console.log

```
const dados = await login(usuario.value.trim(), senha.value)
iniciarSessao(dados)
router.replace('/')
```

O replace('/') funciona porque a rota / foi registrada no slide anterior.

TESTE DA ETAPA 5

Entrar, atualizar e sair

Faça

1. Entre com admin e admin123
2. Aperte F5 na Home
3. No DevTools: Application, Session Storage
4. Clique em Sair

Esperado

- Home com Olá, Administrador
- Depois do F5 o nome continua lá
- A chave sessao guarda token e usuário
- Sair apaga a chave e volta ao login

Tem um furo: com a sessão apagada, digite localhost:5173/ na barra. A Home abre mesmo sem login. A etapa 6 resolve isso.

ETAPA 6 DE 7

Rotas protegidas

Um guard fica na porta e só deixa passar quem tem sessão.

Pronto quando: abrir / sem estar logado leva ao /login

CONCEITOS DA ETAPA 6

Um porteiro para as rotas

meta

Informação extra pendurada na rota, como requiresAuth: true. O Vue Router não a usa sozinho; nós a lemos.

Navigation guard

Função que roda antes de cada navegação e decide: liberar ou redirecionar.

to

A rota de destino. Com to.meta.requiresAuth lemos a marca colocada nela.

Guard não é segurança

O front-end pode ser burlado. Quem protege os dados de verdade é a API, validando o token em cada chamada.

ETAPA 6 · ROUTER/INDEX.JS (1 DE 2)

Marcando a rota protegida

NA ROTA DA HOME, ACRESCENTE A LINHA META

```
{
  path: '/',
  name: 'home',
  component: () ⇒ import('@/views/HomeView.vue'),
  meta: { requiresAuth: true },
},
```

meta marca a rota como protegida.

/login não tem meta e continua livre.

A marca sozinha não faz nada: o guard vai lê-la.

ETAPA 6 · ROUTER/INDEX.JS (2 DE 2)

O guard que decide quem entra

IMPORT NO TOPO E O GUARD ANTES DE export default

```
import { session } from '@/stores/session'

router.beforeEach((to) => {
  if (to.meta.requiresAuth && !session.token) {
    return { name: 'login' }
  }
})
```

beforeEach roda antes de toda navegação.

return { name } redireciona para outra rota.

Sem return a navegação segue normalmente.

TESTE DA ETAPA 6

Tentar entrar sem sessão

Faça

1. Clique em Sair
2. Digite localhost:5173/ na barra
3. Entre com admin e admin123
4. Apague a chave sessao no DevTools e aperte F5

Esperado

- Sem sessão, a barra vai para /login
- Depois do login a Home abre normalmente
- Sem a chave, o F5 volta para o login

Apagar uma chave é fácil para qualquer pessoa. Por isso a proteção real dos dados precisa estar na API.

ETAPA 7 DE 7

Login de verdade

Trocamos a função de teste por uma chamada fetch à API Spring. A tela não muda uma linha.

Pronto quando: o usuário cadastrado no banco consegue entrar

CONCEITO DA ETAPA 7

O contrato com a API

REQUISIÇÃO

```
POST /api/auth/login
Content-Type: application/json

{ "usuario": "admin",
  "senha": "admin123" }
```

RESPOSTA 200

```
{ "token": "eyJhbGciOi...",
  "usuario": {
    "nome": "Administrador" } }
```

200 · credenciais válidas: a API devolve token e usuário.

401 · credenciais recusadas: a API devolve uma mensagem de erro.

ETAPA 7 · SERVICES/AUTH.JS

login() com fetch

SUBSTITUA A FUNÇÃO DE TESTE

```
export async function login(usuario, senha) {
  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usuario, senha }),
  })
  if (!response.ok) throw new Error('Usuário ou senha inválidos.')
  return response.json()
}
```

fetch envia a requisição e devolve uma Promise.

JSON.stringify transforma o objeto em texto JSON.

response.ok é verdadeiro para status de 200 a 299.

TESTE DA ETAPA 7

Conversando com a API de verdade

Antes de testar

- vendas-api rodando na porta 8080
- Tabela usuario criada, com o admin cadastrado
- Proxy /api no vite.config.js apontando para o 8080

Faça e observe

1. Entre com uma senha errada: alerta de erro
2. Entre com admin e admin123: Home
3. DevTools, aba Network: veja o POST, o Payload e a Response com o token

Desafio: pare a API e tente entrar. Que mensagem aparece? Ela ajuda o usuário?

FLUXO COMPLETO

O caminho de um login

1

Formulário

O usuário digita e entrar() valida os campos.

2

Serviço

login() em auth.js monta o POST com fetch.

3

API

O Spring confere as credenciais e devolve token e nome.

4

Sessão

iniciarSessao() guarda os dados em session e no sessionStorage.

5

Rota

replace('/') leva à Home e o guard libera, pois há token.

6

Sair

encerrarSessao() limpa tudo e o guard volta a bloquear.

REVISÃO

Glossário da aula

Componente
Arquivo .vue com template, script e style

Reatividade
A tela se atualiza quando os dados mudam

ref e reactive
Criam dados que o Vue observa

v-model
Liga um campo a uma variável

Promise
Resultado futuro de uma operação demorada

async e await
Esperam uma Promise sem travar a tela

Token (JWT)
Comprovante emitido pela API após o login

sessionStorage
Memória do navegador até fechar a aba

Guard
Função que libera ou redireciona uma rota

SPA
Página única que troca de tela sem recarregar

Proxy do Vite
Repassa /api ao back-end em desenvolvimento

Camada de serviço
Funções que escondem os detalhes de HTTP

PARA EVOLUIR

Desafios para depois da aula

Voltar para onde queria ir

Guardar a rota de origem em ?redirect= e usá-la depois do login.

Mensagens de erro melhores

Diferenciar senha errada de API fora do ar, trocando o Failed to fetch por um texto amigável.

Enviar o token

Mandar o header Authorization: Bearer nas chamadas, para a API saber quem está pedindo.

Sessão expirada

Quando a API responder 401 com token, limpar a sessão e voltar ao login.

Próxima aula: layout com menu lateral e as telas de Produto e Venda.